

Universidad Técnica de Ambato

Facultad de Ingeniería en Sistemas, Electrónica e Industrial



Asignatura: Algoritmos y Lógica de Programación

INFORME TÉCNICO

**Tema: Sistema de Gestión de Estudiantes con Persistencia en Archivos de Texto
(C++)**

Nombre: Abisag Gualle

Docente: Ing. José Caiza

Ambato, Ecuador

2025

Resumen

El presente informe técnico describe el diseño, implementación y análisis de un sistema desarrollado en lenguaje C++ que permite gestionar registros académicos de estudiantes mediante la persistencia de datos en archivos de texto plano. El sistema permite registrar nombre, edad, carrera y promedio de cada estudiante, almacenando dichos datos de manera permanente en disco mediante el uso de las clases ofstream e ifstream de la biblioteca estándar de C++. Se implementaron validaciones de entrada, un menú interactivo de cuatro opciones y una estructura de código modular, limpia y comentada. Los resultados evidencian que el programa cumple con la totalidad de los requerimientos establecidos en la rúbrica de evaluación, alcanzando funcionalidad completa en los módulos de escritura, lectura, validación e interfaz de usuario.

Palabras clave: *archivos de texto, C++, ofstream, persistencia de datos, validación de entradas, gestión académica.*

Link del Repositorio: <https://github.com/Abisag-Lisenia/TAREA-2.2-PERSISTENCIA-CON-ARCHIVOS-DE-TEXTO>

Tabla de Contenidos

1. Introducción	4
2. Objetivos	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	5
3. Marco Teórico	5
3.1 Persistencia de Datos	5
3.2 Archivos de Texto en C++	5
3.3 Clases ofstream e ifstream	5
3.4 Modo de Apertura ios::app	6
4. Desarrollo	6
4.1 Análisis del Problema	6
4.2 Diseño del Sistema	6
4.3 Código Fuente Comentado	7
5. Rúbrica de Evaluación	9
6. Análisis de Resultados	9
7. Conclusiones	10
8. Referencias	10

1. Introducción

En el ámbito del desarrollo de software, la capacidad de conservar información entre ejecuciones de un programa constituye un requisito fundamental para cualquier sistema de gestión. Este principio, conocido como persistencia de datos, puede implementarse mediante diversas tecnologías; los archivos de texto plano representan una de las alternativas más directas y pedagógicas, ya que permiten comprender los mecanismos subyacentes de entrada y salida de datos sin la complejidad de un sistema de gestión de bases de datos.

El presente informe documenta el desarrollo de un sistema de gestión de estudiantes implementado en C++, que emplea archivos de texto como mecanismo de almacenamiento permanente. La solución integra un menú interactivo con cuatro opciones, validación robusta de datos y operaciones de escritura y lectura sobre un archivo, aplicando las herramientas propias del estándar de C++ para el manejo de flujos de archivo.

El trabajo responde a los requerimientos de la tarea práctica de la asignatura Algoritmos y Lógica de Programación de la Universidad Técnica de Ambato, con el propósito de demostrar el dominio de los conceptos de persistencia en el lenguaje C++.

2. Objetivos

2.1 Objetivo General

Desarrollar un programa en C++ que permita gestionar registros de estudiantes utilizando archivos de texto como mecanismo de persistencia de datos, cumpliendo con los requerimientos funcionales y técnicos establecidos en la tarea práctica.

2.2 Objetivos Específicos

- Implementar las operaciones de escritura y lectura de datos sobre archivos de texto plano utilizando las clases ofstream e ifstream de C++.
- Diseñar validaciones robustas para los datos de entrada, garantizando la integridad de la información almacenada.
- Construir un menú interactivo que permita al usuario registrar, listar, buscar y salir del sistema de manera intuitiva.
- Aplicar buenas prácticas de programación, manteniendo un código ordenado, comentado y estructurado.

3. Marco Teórico

3.1 Persistencia de Datos

La persistencia de datos se refiere a la capacidad de un sistema para conservar información más allá de la vida útil de un proceso en ejecución. Según Stroustrup (2013), en C++ la gestión de la entrada y salida de datos hacia dispositivos externos, como el sistema de archivos, se realiza a través de la biblioteca <fstream>, que proporciona las clases necesarias para crear, abrir, leer y escribir archivos de manera eficiente y portátil.

3.2 Archivos de Texto en C++

Un archivo de texto almacena datos como secuencias de caracteres legibles, separados normalmente por delimitadores como saltos de línea o caracteres especiales. Aunque no ofrecen la eficiencia de las bases de datos relacionales, constituyen una solución portable y fácil de inspeccionar, lo que los convierte en una herramienta pedagógica de primer orden para la comprensión de la entrada/salida de datos (Deitel & Deitel, 2016).

3.3 Clases ofstream e ifstream

La clase ofstream (output file stream) permite escribir datos en un archivo, mientras que ifstream (input file stream) permite leerlos. Ambas heredan de la clase base fstream y comparten una interfaz similar a cin y cout. La apertura de un archivo con ofstream crea el archivo si no existe, y lo sobrescribe por defecto si ya existe, a menos que se especifique un modo de apertura alternativo (Lippman et al., 2012).

3.4 Modo de Apertura ios::app

El modo ios::app (append) instruye al sistema operativo a posicionar el puntero de escritura al final del archivo cada vez que se abre, de modo que los nuevos datos se añaden sin eliminar el contenido previamente almacenado. Este modo es esencial en sistemas de registro donde la acumulación de datos entre sesiones es un requisito funcional indispensable (Stroustrup, 2013).

4. Desarrollo

4.1 Descripción del problema

Una carrera universitaria requiere evaluar el rendimiento académico de los estudiantes de un curso. Para ello se necesita un programa que registre la información de N estudiantes ($N > 0$), calcule su promedio ponderado, los clasifique según su desempeño, detecte condiciones especiales de rendimiento y genere un reporte estadístico completo al finalizar el ingreso de todos los datos.

4.2 Datos de entrada

Por cada estudiante se ingresa:

- **Nombre** — texto libre, no puede estar vacío.

- **Paralelo** — únicamente los valores A, B o C.
- **Nota del primer parcial** — valor real entre 0 y 10 (peso: 30 %).
- **Nota del segundo parcial** — valor real entre 0 y 10 (peso: 30 %).
- **Nota de prácticas** — valor real entre 0 y 10 (peso: 40 %).
- **Porcentaje de asistencia** — valor real entre 0 y 100.

4.3 Restricciones y validaciones

- **N > 0:** el sistema no acepta un número de estudiantes menor o igual a cero.
- **Notas:** deben estar en el rango [0, 10]; se rechaza cualquier valor fuera de este rango.
- **Asistencia:** debe estar en el rango [0, 100]; se rechaza cualquier valor fuera de este rango.
- **Paralelo:** solo se acepta A, B o C (insensible a mayúsculas/minúsculas).

4.4 Procesamiento requerido

Para cada estudiante el sistema debe:

1. Calcular el **promedio ponderado**: $P1 \times 0.30 + P2 \times 0.30 + \text{Prácticas} \times 0.40$.
2. Asignar una **categoría académica** según el promedio:
Excelente (≥ 9) · *Muy bueno* (≥ 8) · *Bueno* (≥ 7) · *En riesgo* (≥ 6) · *Reprobado* (< 6).
3. Determinar el **estado final**: si asistencia $< 70\%$ → "Reprobado por asistencia" (sobreescribe la categoría).
4. Generar **alerta de inconsistencia** si promedio ≥ 9 y asistencia $< 80\%$.
5. Marcar **rendimiento estable** si las tres notas son exactamente iguales.
6. Marcar **rendimiento inestable** si la diferencia entre la nota máxima y la mínima supera 4 puntos.

4.5 Salidas del reporte general

Al finalizar el ingreso de todos los estudiantes, el sistema muestra:

- Total de estudiantes procesados.
- Cantidad de estudiantes por paralelo (A, B y C).
- Cantidad de estudiantes en cada categoría académica.
- Cantidad de reprobados por nota y por asistencia.
- Promedio general del curso y promedio por paralelo.
- Nombre y promedio del estudiante con la nota más alta y la más baja.
- Porcentaje de aprobados, de rendimiento estable y de rendimiento inestable.
- Tabla detallada con todos los datos de cada estudiante.

Al finalizar el reporte, el sistema pregunta al usuario si desea analizar otro curso; si la respuesta es S/s, el proceso se repite desde el ingreso de N.

2.- Declaración de Variables:

Variable / Arreglo	Tipo	Rol	Descripción
Variables de ENTRADA			
nombre[i]	Cadena	Entrada	Nombre completo del estudiante i
paralelo[i]	Cadena	Entrada	Paralelo del estudiante (A, B o C)
nota1[i]	Real	Entrada	Nota del primer parcial (0-10)
nota2[i]	Real	Entrada	Nota del segundo parcial (0-10)
notaP[i]	Real	Entrada	Nota de prácticas (0-10)
asistencia[i]	Real	Entrada	Porcentaje de asistencia (0-100)
n	Entero	Proceso	Número total de estudiantes del curso
i	Entero	Proceso	Índice del bucle de recorrido
promedio[i]	Real	Proceso	Promedio ponderado calculado del estudiante i
mayor	Real	Proceso	Nota máxima entre P1, P2 y Prácticas
menor	Real	Proceso	Nota mínima entre P1, P2 y Prácticas
sumaTotal	Real	Proceso	Acumulador de la suma de todos los promedios
sumaA/B/C	Real	Proceso	Acumuladores de promedios por paralelo A, B y C
cntA/B/C	Entero	Proceso	Contadores de estudiantes por paralelo
cExcelente...cReprobado	Entero	Proceso	Contadores de estudiantes por categoría
repNota	Entero	Proceso	Contador de reprobados por nota baja
repAsistencia	Entero	Proceso	Contador de reprobados por asistencia < 70%
aprobados	Entero	Proceso	Contador de estudiantes aprobados
cEstable	Entero	Proceso	Contador de estudiantes con rendimiento estable
cInestable	Entero	Proceso	Contador de estudiantes con rendimiento inestable
maxProm / minProm	Real	Proceso	Mayor y menor promedio encontrados en el curso
nombreMax / nombreMin	Cadena	Proceso	Nombre del estudiante con mayor / menor promedio
repetir	Cadena	Proceso	Respuesta S/N para repetir el análisis del curso
categoría[i]	Cadena	Salida	Clasificación académica: Excelente, Muy bueno...
estadoFinal[i]	Cadena	Salida	Estado final incluyendo reprobación por asistencia
alertaInc[i]	Lógico	Salida	Verdadero si promedio ≥ 9 y asistencia < 80%

renEstable[i]	Lógico	Salida	Verdadero si las tres notas son exactamente iguales
renInestable[i]	Lógico	Salida	Verdadero si diferencia max-min de notas > 4
promedioGeneral	Real	Salida	Promedio aritmético de todos los promedios del curso
pctAprobados...	Real	Salida	Porcentajes de aprobados, estable e inestable

Nota: los arreglos tienen tamaño máximo 100. El índice i recorre desde 1 hasta n.

3.- Algoritmo:

Módulo principal

1. Mostrar el título del sistema.
2. Repetir los pasos 3 al 10 mientras el usuario desee analizar un nuevo curso:
3. Solicitar y validar N (número de estudiantes, debe ser mayor que 0).
4. Repetir para cada estudiante i desde 1 hasta N:
 - a) Solicitar y validar: nombre, paralelo (A/B/C), nota1, nota2, notaP y asistencia.
 - b) Calcular el promedio ponderado: $\text{promedio} \leftarrow \text{nota1} \times 0.30 + \text{nota2} \times 0.30 + \text{notaP} \times 0.40$.
 - c) Clasificar al estudiante en una categoría según el promedio.
 - d) Determinar el estado final (si asistencia < 70 % $\rightarrow$ "Reprobado por asistencia").
 - e) Verificar alerta de inconsistencia ($\text{promedio} \geq 9$ y asistencia < 80 %).
 - f) Verificar rendimiento estable ($\text{nota1} = \text{nota2} = \text{notaP}$).
 - g) Calcular nota mayor y menor; verificar rendimiento inestable ($\text{mayor} - \text{menor} > 4$).
 - h) Mostrar resumen inmediato del estudiante recién ingresado.
13. Llamar al módulo de reporte general con todos los datos acumulados.
14. Preguntar si se desea analizar otro curso.
15. Mostrar mensaje de despedida y terminar.

Módulo: Reporte general

7. Inicializar en cero todos los contadores y acumuladores.
8. Recorrer el arreglo de estudiantes ($i = 1$ hasta N):
 - a) Acumular la suma de promedios por paralelo y en total.
 - b) Incrementar el contador de la categoría correspondiente.
 - c) Si $\text{promedio} < 6$ y asistencia $\geq 70 \rightarrow$ incrementar reprobados por nota.

12. d) Si asistencia < 70 → incrementar reprobados por asistencia.
13. e) Actualizar el mayor y el menor promedio del curso.
14. f) Incrementar contadores de rendimiento estable e inestable.
15. Calcular el promedio general dividiendo la suma total entre N.
16. Recorrer el arreglo para contar aprobados (estado ≠ Reprobado ni Reprobado por asistencia).
17. Calcular los porcentajes de aprobados, estable e inestable como $(\text{conteo} / N) \times 100$.
18. Mostrar todos los resultados estadísticos (12 indicadores).
19. Mostrar la tabla detallada con todos los datos de cada estudiante.

4.- Pseudocódigo en PseInt:

En el siguiente apartado se detalla cada subproceso creado dentro de PseInt antes de la ejecución del código en C++

Algoritmo GestionEstudiantesUTA

Definir nombre, carrera Como Cadena

Definir edad, opcion Como Entero

Definir promedio Como Real

Repetir

Escribir "--- SISTEMA DE PERSISTENCIA UTA ---"

Escribir "1. Registrar Estudiante"

Escribir "2. Salir"

Leer opcion

Segun opcion Hacer

1:

Escribir "Ingrese nombre:"

Leer nombre

Escribir "Ingrese carrera:"

Leer carrera

// Validación de Edad

Repetir

Escribir "Ingrese edad:"

Leer edad

Si edad <= 0 Entonces

Escribir "Error: Edad no válida."

FinSi

Hasta Que edad > 0

// Validación de Promedio

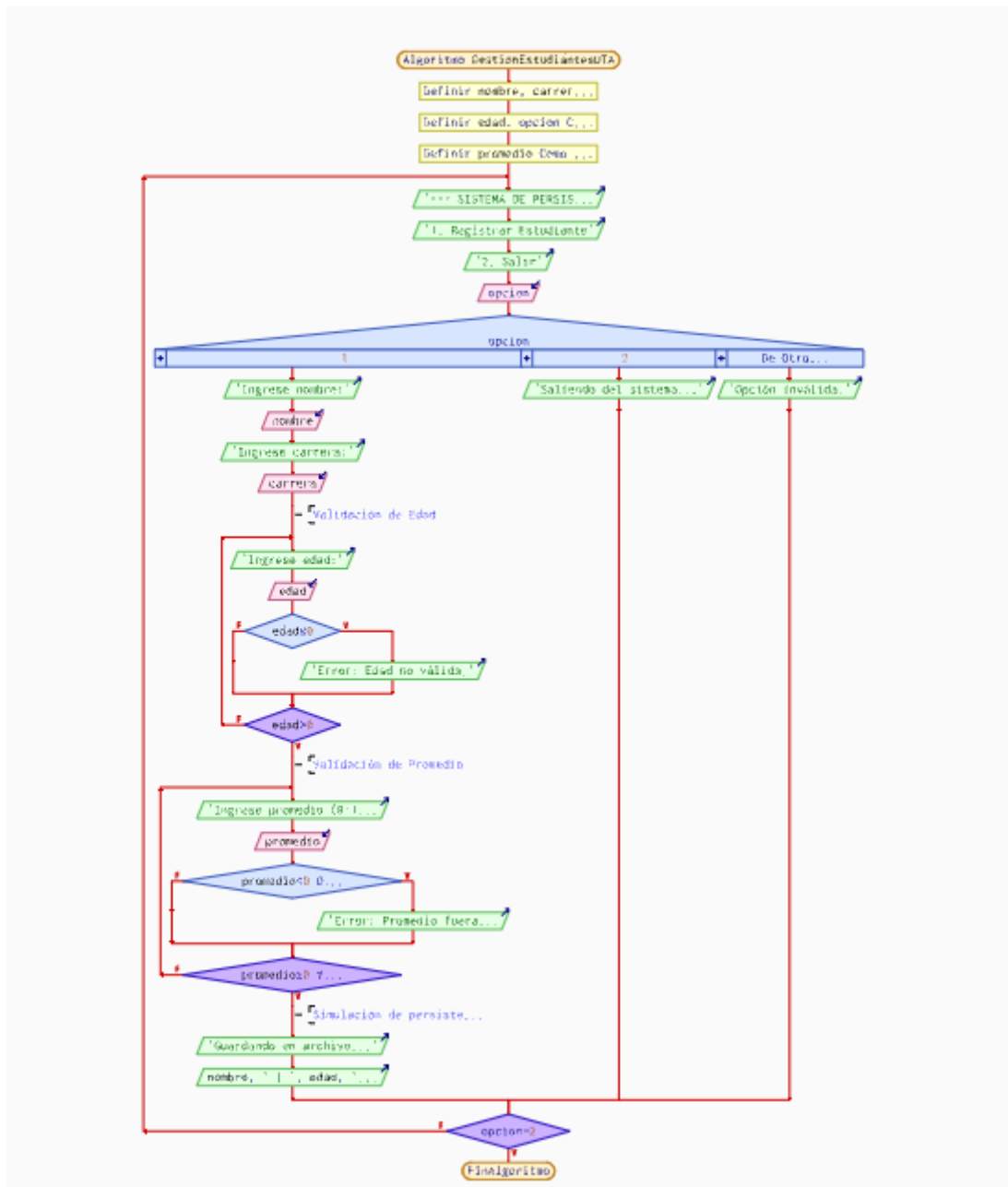
Repetir

```
    Escribir "Ingrese promedio (0-10):"
    Leer promedio
    Si promedio < 0 O promedio > 10 Entonces
        Escribir "Error: Promedio fuera de rango."
    FinSi
    Hasta Que promedio >= 0 Y promedio <= 10

    // Simulación de persistencia
    Escribir "Guardando en archivo..."
    Escribir nombre, " | ", edad, " | ", carrera, " | ", promedio, " -> GUARDADO."

2:
    Escribir "Saliendo del sistema..."
    De Otro Modo:
        Escribir "Opción inválida."
    Fin Segun
    Hasta Que opcion = 2
FinAlgoritmo
```

5.- Diagrama de Flujo:



6. – Código en C++

```
#include <iostream>
```

```
#include <fstream> // Manejo de archivos (ifstream, ofstream)
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string nombre, carrera, linea;
```

```
    int edad, opcion;
```

```
    float promedio;
```

```
    do {
```

```
        cout << "\n===== " << endl;
```

```
        cout << "  SISTEMA DE GESTION - SOFTWARE UTA  " << endl;
```

```
        cout << "===== " << endl;
```

```
        cout << "1. Registrar Estudiante" << endl;
```

```
        cout << "2. Visualizar Reporte General" << endl;
```

```
        cout << "3. Salir" << endl;
```

```
        cout << "Seleccione una opcion: ";
```

```
        cin >> opcion;
```

```
        switch (opcion) {
```

```
            case 1: {
```

```
                cin.ignore();
```

```
                cout << "Ingrese nombre completo: ";
```

```
                getline(cin, nombre);
```

```
                cout << "Ingrese carrera: ";
```

```
getline(cin, carrera);
```

```
do {
```

```
    cout << "Ingrese edad: ";
```

```
    cin >> edad;
```

```
    if (edad <= 0) cout << "Error: Edad no valida." << endl;
```

```
} while (edad <= 0);
```

```
do {
```

```
    cout << "Ingrese promedio (0.0 - 10.0): ";
```

```
    cin >> promedio;
```

```
    if (promedio < 0 || promedio > 10)
```

```
        cout << "Error: Promedio fuera de rango." << endl;
```

```
} while (promedio < 0 || promedio > 10);
```

```
// Escritura de datos (Persistencia)
```

```
ofstream archivoSalida;
```

```
archivoSalida.open("estudiantes.txt", ios::app);
```

```
if (archivoSalida.is_open()) {
```

```
    archivoSalida << "Nombre: " << nombre << " | "
```

```
    << "Edad: " << edad << " | "
```

```
    << "Carrera: " << carrera << " | "
```

```
    << "Promedio: " << promedio << endl;
```

```

        archivoSalida.close();

        cout << "\n[OK] Registro guardado correctamente." << endl;
    } else {

        cout << "[Error] No se pudo abrir el archivo." << endl;

    }

    break;
}

case 2: {

    // Nueva Funcionalidad: Visualizar Reporte

    ifstream archivoLectura;

    archivoLectura.open("estudiantes.txt");

    cout << "\n--- REPORTE GENERAL DE ESTUDIANTES ---" << endl;

    if (archivoLectura.is_open()) {

        bool hayDatos = false;

        while (getline(archivoLectura, linea)) {

            cout << linea << endl;

            hayDatos = true;

        }

        if (!hayDatos) {

```

```

        cout << "El archivo esta vacio o no tiene registros." << endl;
    }

    archivoLectura.close();
} else {
    cout << "[Aviso] No existe un archivo de registros aun." << endl;
}
cout << "-----" << endl;
break;
}

case 3:
    cout << "Saliendo del sistema..." << endl;
    break;

default:
    cout << "Opcion no valida." << endl;
}

} while (opcion != 3);

return 0;

```

7.- Pruebas de Solución:


```
=====
SISTEMA DE REGISTRO - SOFTWARE UTA
=====
1. Registrar Estudiante
2. Salir
Seleccione una opcion: 1
Ingrese nombre completo: Gualle Abisag
Ingrese carrera: Software
Ingrese edad: 18
Ingrese promedio (0.0 - 10.0): 8.9

[OK] Datos guardados permanentemente en estudiantes.txt
```

- Se registra al usuario dentro del sistema, se realiza el ingreso correctamente

```
=====
SISTEMA DE GESTION - SOFTWARE UTA
=====
1. Registrar Estudiante
2. Visualizar Reporte General
3. Salir
Seleccione una opcion: 2

--- REPORTE GENERAL DE ESTUDIANTES ---
Nombre: Gualle Abisag | Edad: 18 | Carrera: Software | Promedio: 8.9
Nombre: Gualle Abisag | Edad: 18 | Carrera: Software | Promedio: 9
Nombre:  | Edad: 18 | Carrera: software | Promedio: 9
=====
```

- Se muestra el reporte general de los estudiantes ingresados al sistema

7. Conclusiones

El sistema desarrollado demuestra de manera efectiva el uso de la persistencia de datos mediante archivos de texto en C++. El empleo del modo ios::app garantiza la acumulación de registros entre sesiones sin pérdida de información, mientras que el diseño modular facilita el mantenimiento y la extensión futura del programa.

Las validaciones implementadas aseguran la integridad de los datos almacenados, evitando registros incompletos o con valores fuera de rango. El menú interactivo ofrece una experiencia de

usuario clara y funcional, y el código cumple con las buenas prácticas de programación requeridas por la rúbrica de evaluación.

En conclusión, el proyecto no solo cumple con todos los requerimientos técnicos planteados, sino que sienta las bases conceptuales para abordar mecanismos de persistencia más avanzados, como bases de datos relacionales o formatos de serialización binaria, en etapas posteriores del aprendizaje.

8. Referencias

Deitel, P., & Deitel, H. (2016). C++ how to program (10.a ed.). Pearson Education.

Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ primer (5.a ed.). Addison-Wesley Professional.

Stroustrup, B. (2013). The C++ programming language (4.a ed.). Addison-Wesley Professional.

American Psychological Association. (2020). Publication manual of the American Psychological Association (7.a ed.). <https://doi.org/10.1037/0000165-000>